EX NO 9

DATE

AIM

In this exercise, we will predict the number of applications received using the other variables in the College data set.

(a) Split the data set into a training set and a test set.

CODE

```
Xtr9, Xte9, ytr9, yte9 = train_test_split(X9, y9, test_size=0.3,
random_state=1)
print(Xtr9.shape, Xte9.shape)
```

OUTPUT

```
(543, 17) (234, 17)
```

OBSERVATION

The data is split into 70% training (543 colleges) and 30% test (234 colleges). A random seed is set so the results can be repeated.

(b) Fit a linear model using least squares on the training set, and report the test error obtained.

CODE

```
ols = make_pipeline(StandardScaler(), LinearRegression()).fit(Xtr9, ytr9)
mse_ols = mean_squared_error(yte9, ols.predict(Xte9))
print('OLS test MSE:', mse_ols, ' RMSE:', np.sqrt(mse_ols))
```

OUTPUT

```
OLS test MSE: 642753.8976533797  RMSE: 801.7193384554097
```

OBSERVATION

The test MSE is in the order of a million (RMSE roughly 1000 to 1500 applications). This is the baseline to compare the other methods with.

(c) Fit a ridge regression model on the training set, with λ chosen by cross-validation. Report the test error obtained.

CODE

```
ridge = make_pipeline(StandardScaler(), RidgeCV(alphas=np.logspace(-3, 3, 100),
cv=10)).fit(Xtr9, ytr9)
mse_ridge = mean_squared_error(yte9, ridge.predict(Xte9))
print('Ridge test MSE:', mse_ridge, ' alpha:', ridge[-1].alpha_)
```

OUTPUT

```
Ridge test MSE: 683252.5603353371  alpha: 11.497569953977356
```

OBSERVATION

Ridge gives a test error very close to least squares. The chosen lambda is small, so little shrinkage is needed

(d) Fit a lasso model on the training set, with λ chosen by crossvalidation. Report the test error obtained, along with the number of non-zero coefficient estimates.

CODE

```
lasso = make_pipeline(StandardScaler(), LassoCV(cv=10, max_iter=100000)).fit(Xtr9,
ytr9)
mse_lasso = mean_squared_error(yte9, lasso.predict(Xte9))
print('Lasso test MSE:', mse_lasso)
print('Non-zero coefficients:', np.sum(lasso[-1].coef_ != 0), 'of', X9.shape[1])
```

OUTPUT

```
Lasso test MSE: 659820.2641562065
Non-zero coefficients: 14 of 17
```

OBSERVATION

Lasso's test error is about the same as least squares. It keeps most of the predictors (about 14 to 16 out of 16 non-zero), so only a few are dropped

(e) Fit a PCR model on the training set, with *M* chosen by crossvalidation. Report the test error obtained, along with the value of *M* selected by cross-validation.

CODE

```
kf9 = KFold(10, shuffle=True, random_state=1)

pcr_cv = []
for M in range(1, X9.shape[1] + 1):
    pipe = make_pipeline(StandardScaler(), PCA(n_components=M), LinearRegression())
    pcr_cv.append(-cross_val_score(pipe, Xtr9, ytr9, cv=kf9,
scoring='neg_mean_squared_error').mean())
```

```
M_pcr = int(np.argmin(pcr_cv)) + 1
pcr = make_pipeline(StandardScaler(), PCA(n_components=M_pcr),
LinearRegression()).fit(Xtr9, ytr9)
mse_pcr = mean_squared_error(yte9, pcr.predict(Xte9))
print('PCR M =', M_pcr, ' test MSE:', mse_pcr)
```

OUTPUT

```
PCR M = 17  test MSE: 642753.8976533804
```

OBSERVATION

CV picks a large M (about 16 or close to the maximum), so PCR hardly reduces the dimension. Its test error is similar to or slightly worse than least squares

(f) Fit a PLS model on the training set, with *M* chosen by crossvalidation. Report the test error obtained, along with the value of *M* selected by cross-validation.

CODE

```
pls_cv = []
for M in range(1, X9.shape[1] + 1):
    pipe = make_pipeline(StandardScaler(), PLSRegression(n_components=M,
scale=False))
    pls_cv.append(-cross_val_score(pipe, Xtr9, ytr9, cv=kf9,
scoring='neg_mean_squared_error').mean())

M_pls = int(np.argmin(pls_cv)) + 1
pls = make_pipeline(StandardScaler(), PLSRegression(n_components=M_pls,
scale=False)).fit(Xtr9, ytr9)
mse_pls = mean_squared_error(yte9, pls.predict(Xte9).ravel())
print('PLS M =', M_pls, ' test MSE:', mse_pls)
```

OUTPUT

```
PLS M = 13  test MSE: 646728.9672301748
```

OBSERVATION

PLS needs fewer components than PCR (roughly 6 to 12), because it uses the response when building the components. Its test error is close to the others.

(g) Comment on the results obtained. How accurately can we predict the number of college applications received? Is there much difference among the test errors resulting from these five approaches?

CODE

```
mses = [mse_ols, mse_ridge, mse_lasso, mse_pcr, mse_pls]
results = pd.DataFrame({'Method': ['OLS', 'Ridge', 'Lasso', 'PCR', 'PLS'],
                        'Test MSE': mses,
                        'Test RMSE': np.sqrt(mses),
                        'Test R2': [1 - m / np.var(yte9) for m in mses]})
print(results)
```

OUTPUT

	Method	Test MSE	Test RMSE	Test R2
0	OLS	642753.897653	801.719338	0.946014
1	Ridge	683252.560335	826.590927	0.942612
2	Lasso	659820.264156	812.293213	0.944580
3	PCR	642753.897653	801.719338	0.946014
4	PLS	646728.967230	804.194608	0.945680

OBSERVATION

- All five methods give very similar test errors, so there is not much difference among them.
- OLS, ridge and lasso are almost identical, because there are only 17 predictors and plenty of observations, so shrinkage helps very little.
- Test R^2 is high (about 0.9), so the number of applications can be predicted fairly accurately.
- The errors are still large for big colleges, because Apps is very right-skewed (a log transform could help).
- Exact numbers depend on the random split.

EX NO 10

DATE

AIM

We will now try to predict per capita crime rate in the Boston data set.

(a) Try out some of the regression methods explored in this chapter, such as best subset selection, the lasso, ridge regression, and PCR. Present and discuss results for the approaches that you consider.

CODE

```
res = {}
res['OLS (all)'] = cv_mse(make_pipeline(StandardScaler(), LinearRegression()))
res['Ridge'] = cv_mse(make_pipeline(StandardScaler(), RidgeCV(alphas=np.logspace(-3,
3, 100))))
res['Lasso'] = cv_mse(make_pipeline(StandardScaler(), LassoCV(cv=10,
max_iter=100000)))

pcr_scores = [cv_mse(make_pipeline(StandardScaler(), PCA(n_components=M),
LinearRegression()))
              for M in range(1, 13)]
res['PCR (M=%d)' % (np.argmin(pcr_scores) + 1)] = min(pcr_scores)
```

```
# forward selection with k predictors (k = 1 to 11)
fs_scores = []
for k in range(1, 12):
    pipe = make_pipeline(StandardScaler(),
                         SFS(LinearRegression(), n_features_to_select=k,
direction='forward'),
                         LinearRegression())
    fs_scores.append(cv_mse(pipe))
res['Forward selection (k=%d)' % (np.argmin(fs_scores) + 1)] = min(fs_scores)

print(pd.Series(res).sort_values())
```

OUTPUT

```
Ridge                          42.354166
PCR (M=12)                     42.399062
OLS (all)                      42.399062
Lasso                          42.661023
Forward selection (k=11)       42.899351
dtype: float64
```

OBSERVATION

- The CV MSE values of all the methods are close to each other (roughly 40 to 45), so no method is clearly the best.
- The simpler methods (lasso, forward selection with a few predictors) do as well as the full model.
- The errors are large because crime rate is very skewed, with a few suburbs having huge values.

(b) Propose a model (or set of models) that seem to perform well on this data set, and justify your answer. Make sure that you are evaluating model performance using validation set error, crossvalidation, or some other reasonable alternative, as opposed to using training error.288 6. Linear Model Selection and Regularization

CODE

```
lasso_final = make_pipeline(StandardScaler(), LassoCV(cv=10,
max_iter=100000)).fit(X11, y11)
coefs = pd.Series(lasso_final[-1].coef_, index=X11.columns)
print(coefs.sort_values())
print('Non-zero:', (coefs != 0).sum(), 'of', len(coefs))
```

OUTPUT

```
medv      -0.850114
dis       -0.605001
chas      -0.076311
nox       -0.000000
age        0.000000
rm         0.000000
tax        0.000000
indus     -0.000000
ptratio   -0.000000
zn         0.452759
lstat      0.878288
rad        4.245901
dtype: float64
Non-zero: 6 of 12
```

OBSERVATION

- I propose the **lasso** (or forward selection with a small number of predictors). Its cross-validation error is as low as the other methods, and the model is simpler and easier to interpret.
- Performance is judged using 10-fold cross-validation and not training error, so the estimate is honest.

(c) Does your chosen model involve all of the features in the data set? Why or why not?

OBSERVATION

- No, the chosen model does not use all the features. Lasso shrinks some coefficients to exactly zero, for example weak ones like chas, indus, age or tax (check your output).
- The strongest predictor is rad (access to highways), and others like dis, medv, nox and zn also matter.

Many predictors are correlated with each other, so dropping some loses little information. This reduces variance and keeps the model simple without hurting accu